

SDN ARP-Aware Traffic Control using POX and Mininet

Project Overview

This project demonstrates **Software Defined Networking (SDN)** with a focus on **ARP-aware traffic control** using the POX controller and Mininet emulator.

The controller enforces network policies by installing OpenFlow rules while leveraging ARP for host discovery and communication setup.

Objectives

- Understand SDN architecture (Control Plane vs Data Plane)
 - Demonstrate ARP role in network communication
 - Implement centralized control using POX
 - Apply OpenFlow match-action rules
 - Block communication between selected hosts (h1 → h3)
-

Network Topology

- 1 Switch (s1)
 - 3 Hosts:
 - h1 → 10.0.0.1
 - h2 → 10.0.0.2
 - h3 → 10.0.0.3
-

Technologies Used

- Mininet
- POX Controller
- OpenFlow
- Ubuntu Linux

SDN Architecture

- Control Plane → POX Controller
 - Data Plane → Open vSwitch
-

ARP Handling in SDN

In traditional networks, ARP (Address Resolution Protocol) is used to map IP addresses to MAC addresses.

In this project:

- ARP is used for **host discovery**
 - Hosts use ARP before communication begins
 - The controller observes packet flow after ARP resolution
 - Policies are enforced centrally using OpenFlow rules
-

Controller Logic

The controller installs a **high-priority OpenFlow DROP rule**:

- Source IP: 10.0.0.1 (h1)
 - Destination IP: 10.0.0.3 (h3)
 - Action: DROP
-

Packet Behavior

Allowed

- h2 → h3 → Works
- h1 → h2 → Works

Blocked

- h1 → h3 → Blocked (100% packet loss)

Why No "Destination Host Unreachable"?

Packets are **silently dropped** by the switch due to the DROP rule. No ICMP error is generated → results in packet loss instead of unreachable message.

How to Run

Start Controller

```
cd ~/pox ./pox.py forwarding.l2_learning arp_controller
```

Start Mininet

```
sudo mn --topo single,3 --controller=remote,ip=127.0.0.1,port=6633 //inbuilt topology
```

Testing

Ping

```
h1 ping h3 h2 ping h3 pingall
```

Iperf

```
h3 iperf -s -u & h2 iperf -c 10.0.0.3 -u
```

Flow Rules

```
sudo ovs-ofctl dump-flows s1
```

Screenshots

- Ping_blocked.png

```
mininet> h1 ping h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable
From 10.0.0.1 icmp_seq=2 Destination Host Unreachable
From 10.0.0.1 icmp_seq=3 Destination Host Unreachable
From 10.0.0.1 icmp_seq=4 Destination Host Unreachable
From 10.0.0.1 icmp_seq=5 Destination Host Unreachable
From 10.0.0.1 icmp_seq=6 Destination Host Unreachable
From 10.0.0.1 icmp_seq=7 Destination Host Unreachable
From 10.0.0.1 icmp_seq=8 Destination Host Unreachable
From 10.0.0.1 icmp_seq=9 Destination Host Unreachable
^C
--- 10.0.0.3 ping statistics ---
11 packets transmitted, 0 received, +9 errors, 100% packet loss, time 10227ms
pipe 4
```

- Ping_allowed.png

```
mininet> h2 ping h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=4.61 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=1.55 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=0.143 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=0.054 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=0.222 ms
^C
--- 10.0.0.3 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4034ms
rtt min/avg/max/mdev = 0.054/1.315/4.608/1.735 ms
```

- Pingall.png

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 X
h2 -> h1 h3
h3 -> X h2
*** Results: 33% dropped (4/6 received)
```

- Iperf.png

```
mininet> h3 iperf -s -u &
0.0.3 -u[1] 1451
mininet> h2 iperf -c 10.0.0.3 -u

-----
Client connecting to 10.0.0.3, UDP port 5001
Sending 1470 byte datagrams, IPG target: 11215.21 us (kalman adjust)
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.2 port 57472 connected with 10.0.0.3 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-10.0155 sec 1.25 MBytes 1.05 Mbits/sec
[ 1] Sent 896 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer    Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-10.0142 sec 1.25 MBytes 1.05 Mbits/sec 0.011 ms 0/895 (0%)
```

- Flow_table.png

```
vishnu_@LAPTOP-U9CDB8I1M:~$ sudo ovs-ofctl dump-flows s1
[sudo] password for vishnu:
cookie=0x0, duration=179.609s, table=0, n_packets=2, n_bytes=196, priority=300,ip,nw_src=10.0.0.1,nw_dst=10.0.0.3 actions=drop
```

- Controller_logs.png

```
vishnu_@LAPTOP-U0CDB0IM:~/pox$ cd ~/pox
./pox.py forwarding.l2_learning arp_controller
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
INFO:arp_controller:Blocking Controller Started
WARNING:version:POX requires one of the following versions of Python: 3.6 3.7 3.8 3.9
WARNING:version:You're running Python 3.12.
WARNING:version:If you run into problems, try using a supported version.
INFO:core:POX 0.7.0 (gar) is up.
INFO:openflow.of_01:[00-00-00-00-00-01 1] connected
INFO:arp_controller:DROP rule installed for h1 -> h3
- INFO:openflow.of_01:[00-00-00-00-00-01 1] closed
```

Conclusion

This project demonstrates how SDN enables centralized control while ARP supports host discovery. The controller enforces policies using OpenFlow rules.

Author

Vishnu Maddirala